offer has expired. Disappointed, you return to Natter to ask your friend about it, only to discover that someone has somehow managed to post abusive messages to some of your friends, apparently sent by you! You also seem to have posted the same offer link to your other friends.

The appeal of cookies as an API designer is that, once set, the browser will transparently add them to every request. As a client developer, this makes life simple. After the user has redirected back from the login endpoint, you can just make API requests without worrying about authentication credentials. Alas, this strength is also one of the greatest weaknesses of session cookies. The browser will also attach the same cookies when requests are made from other sites that are not your UI. The site you visited when you clicked the link from Polly loaded some JavaScript that made requests to the Natter API from your browser window. Because you're still logged in, the browser happily sends your session cookie along with those requests. To the Natter API, those requests look as if you had made them yourself.

As shown in figure 4.9, in many cases browsers will happily let a script from another website make cross-origin requests to your API; it just prevents them from reading any response. Such an attack is known as *cross-site request forgery* because the malicious site can create fake requests to your API that appear to come from a genuine client.

> **DEFINITION** ***Cross-site request forgery*** (CSRF, pronounced "sea-surf") occurs when an attacker makes a cross-origin request to your API and the browser sends cookies along with the request. The